ZeroWatch — Prototype Build Plan

HackToFuture 4.0
Team AlphaQ

Complete guide for building the ZeroWatch prototype using Claude Code

32h	7	5	6
Build time	Phases	Demo moments	Claude Code prompts

32h	7	5	6	~350
Total build time	Build phases	Demo moments	Claude Code prompts	Lines of core Python

PROTOTYPE PHILOSOPHY

Build the **5 demo-able moments**, not the production system. Every architectural simplification is invisible to a judge during a 5-minute demo. What is not invisible: whether Claude generates a baseline in 3 seconds, whether the score climbs live on the chart, and whether the stolen token is instantly rejected. Those three must work flawlessly.

Production Component	Prototype Replacement	Why Fine for Demo
Apache Kafka	asyncio.Queue	Same decoupled pattern, zero install
Apache Flink	Python background thread	Same z-score logic, handles demo load
Istio EnvoyFilter	FastAPI BaseHTTPMiddleware	Same header injection, single process
ONNX Runtime	scikit-learn in-process	Same Isolation Forest, no export needed
PostgreSQL + TimescaleDB	SQLite via SQLModel	Zero config, persists across restarts
Redis (keep)	**Redis (keep)**	**Real Redis — judges can inspect with redis-cli**
Claude Sonnet API (keep)	**Claude Sonnet API (keep)**	**Real LLM — all 3 innovations visible**

TECH STACK

Layer	Tool / Library	Install Command
Backend API	FastAPI + Uvicorn	`pip install fastapi uvicorn[standard]`
Session store	Redis (Docker)	`docker run -d -p 6379:6379 redis:alpine`
Risk scoring	scikit-learn	`pip install scikit-learn numpy`
LLM layer	Anthropic Python SDK	`pip install anthropic`
DB / ORM	SQLite + SQLModel	`pip install sqlmodel`
Device binding	WebCrypto + IndexedDB	`Built into all modern browsers`
IP / Geo	ip-api.com (free)	`No API key needed`
Device fingerprint	FingerprintJS (CDN)	`One <script> tag in HTML`

Frontend	React + Vite + Recharts	`npm create vite@latest`
Styling	Tailwind CSS	`npm install tailwindcss`
HTTP client	aiohttp	`pip install aiohttp`
Crypto (server)	cryptography	`pip install cryptography`

FILE STRUCTURE

Give this exact structure to Claude Code in prompt 1.

```
zerowatch/
  backend/
    main.py            # FastAPI app — all routes + WebSocket
    middleware.py       # ZeroWatch enforcement (ECDSA + risk score)
    risk_engine.py      # z-score baseline + Isolation Forest scorer
    llm.py             # Claude API — cold-start + narration + intent
    ecdsa_verify.py    # Server-side ECDSA P-256 signature verification
    session_store.py   # Redis helpers (get/set/delete session data)
    models.py          # SQLModel: User, Session, Incident, SignalEvent
    demo_app.py        # Target app being protected (login + dashboard)
    signal_worker.py   # asyncio background worker (Kafka replacement)
    config.py          # ENV: ANTHROPIC_API_KEY, REDIS_URL
  frontend/
    src/
      pages/
        Dashboard.jsx   # Live risk timeline + gauge + heatmap
        Incidents.jsx   # LLM incident reports list
        Sessions.jsx    # Active sessions table
      components/
        RiskGauge.jsx         # SVG dial 0-100 (green/amber/red)
        SignalHeatmap.jsx      # Per-feature z-score cards
        IncidentCard.jsx       # Claude narrative display
        AttackSimulator.jsx    # 5-button demo control panel
      hooks/
        useWebSocket.js        # Live score WebSocket hook
  sdk/
    zw-sdk.js          # Browser SDK (ECDSA + fingerprint + fetch interceptor)
  scripts/
    seed_baseline.py   # Generate 30 days of synthetic baseline data
    train_model.py     # Fit Isolation Forest, save .pkl per user
    demo_attack.py     # Simulate stolen token attack deterministically
  docker-compose.yml   # Redis + (optional) backend + frontend
  .env.example         # ANTHROPIC_API_KEY=sk-ant-...
  README.md
```

BUILD PHASES — COMPLETE IN THIS ORDER

Each phase produces something independently testable. Verify it runs before moving to the next.

1 Foundation — FastAPI + Redis + SQLite + config — 2h

- FastAPI app with /health endpoint returns {"status":"ok","redis":"connected"}
- docker-compose.yml starts Redis on port 6379
- config.py loads ANTHROPIC_API_KEY and REDIS_URL from .env
- SQLModel creates User, Session, Incident, SignalEvent tables in SQLite
- Test: redis-cli ping → PONG | curl localhost:8000/health → 200 OK

2 Browser SDK — ECDSA key pair + fingerprint + signals — 4h

- crypto.subtle.generateKey (P-256, extractable:false) → stored in IndexedDB
- Canvas + WebGL fingerprint hashed with SHA-256 → sessionStorage zw_device_fp
- window.fetch interceptor attaches X-ZW-Signature + X-ZW-Timestamp on every call
- Signal events (WPM, mouse velocity) batch-POSTed to /api/signals every 2 seconds
- Test: DevTools Network — every API call has X-ZW-Signature header visible

3 Risk Engine — z-score baseline + Isolation Forest + Redis — 6h

- UserBaseline dataclass: login_hour, wpm, data_mb, ip_countries, device_fps
- score_session() calculates weighted z-score (ip_country 0.25, device_fp 0.20...)
- Unknown country or device fingerprint → z_score = 5.0 (maximum anomaly)
- Score written to Redis key session:{id}:risk_score with 30s TTL
- asyncio background worker processes signal queue every 3 seconds
- Test: send ip_country=RU → redis-cli get session:test:risk_score → >70

4 LLM Layer — 3 Claude API functions (all async) — 6h

- synthesise_baseline(role, dept, perms) → JSON profile in 3 seconds at user creation
- narrate_incident(user_id, z_scores, baseline, flags) → plain-English report <150 words
- classify_intent(user_id, api_sequence) → one word: normal/recon/staging/exfiltration
- All three functions use asyncio.create_task() — never block the hot request path
- Test: POST /api/users → immediate baseline JSON from Claude in response

5 Middleware + ECDSA Enforcement — 4h

- ZeroWatchMiddleware verifies X-ZW-Signature using stored public key from Redis
- Timestamp check: reject if |now - timestamp| > 30 seconds
- Score 0-30: add X-ZeroWatch-Score header, forward request
- Score 31-70 on sensitive endpoint: return 202 {"action":"mfa_required"}
- Score 71+: return 401, delete session from Redis, async LLM incident narration
- Test: send request without X-ZW-Signature → instant 401 {"error":"missing_signature"}

6 React Dashboard — Live chart + gauge + attack simulator — 8h

- WebSocket hook connects to ws://localhost:8000/ws/risk — reconnects on drop
- RiskGauge: SVG dial, colour changes green→amber→red, animates smoothly
- SignalHeatmap: 6 feature cards with z-score bars (green→red)
- AttackSimulator: 5 buttons (Normal, Change Country, Stolen Token, Export, Full Attack)
- IncidentCard: populates with Claude narrative 3 seconds after session terminated
- Recharts LineChart: last 30 risk score data points with dashed predictive trajectory

7 Demo Polish — Scripts + predictive trajectory + day-1 flow — 2h

- scripts/demo_attack.py --scenario stolen_token runs the full attack deterministically
- Predictive trajectory: linear regression on last 5 scores → projected dashed line
- Pre-emptive MFA badge fires when projected score crosses 70 threshold
- Day-1 flow: create user → Claude baseline visible within 3 seconds
- Practice all 5 demo moments until the sequence flows in under 4 minutes

CLAUDE CODE PROMPTS — USE EXACTLY IN ORDER

Give each prompt to Claude Code in a new session. Wait for the phase to run and verify it works before giving the next prompt. Do not give all six at once.

PROMPT 1 — PROJECT INIT

```
Create a Python project "zerowatch" with FastAPI backend.
backend/ folder contains: main.py, config.py (loads ANTHROPIC_API_KEY +
REDIS_URL from .env), models.py (SQLModel: User, Session, Incident,
SignalEvent tables using SQLite), session_store.py (Redis helpers).
GET /health returns {"status":"ok","redis":"connected"} after ping.
docker-compose.yml: redis:alpine on port 6379.
.env.example: ANTHROPIC_API_KEY=sk-ant-... and REDIS_URL=redis://localhost:6379
Run with: uvicorn backend.main:app --reload
```

PROMPT 2 — BROWSER SDK

```
Create sdk/zw-sdk.js — IIFE, no dependencies.
1. crypto.subtle.generateKey(P-256, extractable:false) → IndexedDB
   database:"ZeroWatchKeys" store:"keys" key:"device-signing-key".
   Load existing key if present instead of regenerating.
2. Canvas 2D text + WebGL UNMASKED_RENDERER_WEBGL → SHA-256 hash
   → sessionStorage "zw_device_fp". Export pubkey → "zw_pubkey".
3. window.fetch interceptor: sign "{session_id}|{METHOD}|{path}|{ts}"
   → attach X-ZW-Signature (base64 DER) + X-ZW-Timestamp headers.
4. Collect keyup WPM + mousemove velocity. Batch POST /api/signals every 2s.
5. On login success: POST pubkey to /api/auth/register-device.
   Expose window.ZeroWatch = {getFingerprint, getPublicKey}.
```

PROMPT 3 — RISK ENGINE

```
Create backend/risk_engine.py with UserBaseline dataclass.
Fields: login_hour_mean/std, wpm_mean/std, data_mb_mean/std,
  ip_countries (set), device_fps (set). Load/save from Redis as JSON.
score_session(user_id, signals) → float 0-100:
  z_score per numerical feature = (value - mean) / std.
  Unknown ip_country or device_fp → z_score = 5.0.
  Weights: ip_country=0.25, device_fp=0.20, data_mb=0.15,
    api_calls=0.12, login_hour=0.10, mouse_velocity=0.10, wpm=0.08.
  Write to Redis "session:{id}:risk_score" TTL 30s.
signal_worker() asyncio function: consume queue, call score_session every 3s.
Also create scripts/seed_baseline.py: 30 synthetic normal sessions.
```

PROMPT 4 — LLM INTEGRATIONS

```
Create backend/llm.py using AsyncAnthropic client.
1. synthesise_baseline(role, dept, access_permissions) -> dict:
   Prompt Claude: return JSON {expected_hours_start, expected_hours_end,
   data_mb_range, expected_endpoints, device_type, api_calls_per_min_range}.
   model=claude-sonnet-4-20250514, max_tokens=500.
2. narrate_incident(user_id, z_scores, baseline, flags) -> str:
   Plain-English incident report under 150 words.
   Include: what happened, why suspicious, recommended action.
   max_tokens=300.
3. classify_intent(user_id, api_sequence: list[str]) -> str:
   Return ONLY: "normal" | "reconnaissance" | "staging" | "exfiltration".
   max_tokens=10. Handle errors gracefully with fallback strings.
```

PROMPT 5 — MIDDLEWARE + ENFORCEMENT

```
Create backend/middleware.py: ZeroWatchMiddleware(BaseHTTPMiddleware).
Skip: /health /api/auth/login /api/auth/register-device /api/signals.
1. Extract session_id from cookie. Pass through if missing.
2. ECDSA check: require X-ZW-Signature + X-ZW-Timestamp.
   Missing → 401 {"error":"missing_signature"}.
   Timestamp older than 30s → 401 {"error":"signature_expired"}.
   Load pubkey from Redis. Verify ECDSA(SHA-256). Fail → 401 session_hijack.
3. Read risk score from Redis. Default 0.
4. 0-30: add X-ZeroWatch-Score header, forward.
   31-70 + SENSITIVE_PATH: 202 {"action":"mfa_required","score":score}.
   71+: 401 {"error":"session_terminated"}. DELETE session from Redis.
        asyncio.create_task(narrate_incident(...)). Push to WebSocket.
SENSITIVE_PATHS = ["/app/transfer", "/app/admin", "/app/export"].
```

PROMPT 6 — REACT DASHBOARD

```
In Vite React project, create with Tailwind CSS:
1. hooks/useWebSocket.js: connect ws://localhost:8000/ws/risk.
   Reconnect on disconnect. Return {score,delta,flags,user,incidents}.
2. RiskGauge.jsx: SVG circle dial, score 0-100,
   green 0-30 / amber 31-70 / red 71-100. Animate on change.
3. SignalHeatmap.jsx: 6 feature cards (login_hour, wpm, data_mb,
   ip_country, device_fp, api_calls). Each shows z-score + colour bar.
4. IncidentCard.jsx: user, timestamp, score, Claude narrative, action badge.
5. AttackSimulator.jsx: 5 buttons: Normal Session / Change Country (Russia) /
   Stolen Token (no signature) / Rapid Export / Full Attack.
   Each POSTs to /api/demo/simulate/{scenario}.
6. Dashboard.jsx: RiskGauge + Recharts LineChart top row,
   SignalHeatmap middle, IncidentCard bottom, AttackSimulator fixed bar.
```

DEMO SCRIPT — 5 MOMENTS IN 4 MINUTES

#	Moment	Time	What to show	What to say
1	**Day-1 cold-start**	0:00–0:45	Create user "Sneha / Junior Analyst / Retail Banking". Claude baseline appears.	*"Claude looked at the role and wrote a behavioral profile. Every bank does this in 3 seconds from ro*
2	**Normal session**	0:45–1:15	Click Normal Session. Gauge stays 4–10. All health bars green.	*"Ten requests, all scored. Zero latency, zero friction for the real use*
3	**Stolen token attack**	1:15–2:30	Click Full Attack. Watch gauge climb live: 15→34→67→95. SESSION TERMINATED.	*"Five signals. Independently. All anomalies hit simultaneo*
4	**Claude incident report**	2:30–3:30	Incident panel populates 3 seconds after termination. Show the Claude-written report.	*"Claude doesn't just give you a number. It gives you a story. ZeroWatch gives a story. 45 m*
5	**ECDSA blocks cookie**	3:30–4:00	Click Stolen Token. Instant 401. Show DevTools: no X-ZW-Signature header.	*"Cookie + JWT + device fingerprint + behavior. Rejected on the first request. No e*

DEMO SETUP — WHAT TO HAVE OPEN

Tab / Window	Content	Purpose
Tab 1 (main)	ZeroWatch React dashboard — fullscreen	Risk gauge + chart + incident panel
Tab 2	Demo app login page (localhost:8001)	Shows the "target" being protected
Tab 3	Browser DevTools → Network tab	Shows X-ZW-Signature on every request
Terminal 1	uvicorn backend.main:app --reload	Shows async LLM calls firing in logs
Terminal 2	redis-cli monitor	Judges can see keys written live (optional)
Terminal 3	python scripts/demo_attack.py --scenario stolen_token	Backup deterministic demo

CRITICAL NOTES — MOST COMMON FAILURE POINTS

WebSocket is the most important thing

The live risk score climbing in real time wins the demo. If the chart updates with a 5-second delay the impact is lost completely. Use FastAPI built-in WebSocket and push every score update immediately — never poll.

Test ECDSA before the demo, not during it

The crypto.subtle.sign() call is synchronous to the fetch interceptor. If IndexedDB fails to load the key (first visit, incognito mode, cleared data), all requests will be missing the signature header. Run the demo in the same browser profile you built it in.

Seed the baseline BEFORE creating demo users

The Isolation Forest needs training data to produce meaningful scores. Run scripts/seed_baseline.py first. Without it, all risk scores will be 0 regardless of what signals you send.

Claude API calls must be async

synthesise_baseline(), narrate_incident(), and classify_intent() must use asyncio.create_task() — never await them inline in a request handler. A synchronous Claude call blocks the entire FastAPI event loop and makes the demo appear frozen for 2–3 seconds.

Have ANTHROPIC_API_KEY ready

The three LLM functions are the most visible innovations. Without a valid API key, cold-start and incident narration both fail silently. Set it in .env before starting the server and verify with a curl to the /api/users endpoint.

QUICK START COMMANDS

```
# 1. Clone + install
git init zerowatch && cd zerowatch
pip install fastapi uvicorn[standard] redis anthropic scikit-learn \
            sqlmodel aiohttp cryptography python-multipart python-dotenv
npm create vite@latest frontend -- --template react
cd frontend && npm install recharts tailwindcss @heroicons/react && cd ..

# 2. Start Redis
docker run -d -p 6379:6379 --name zw-redis redis:alpine

# 3. Set env vars
echo "ANTHROPIC_API_KEY=sk-ant-YOUR_KEY" > .env
echo "REDIS_URL=redis://localhost:6379"  >> .env

# 4. Seed baseline + train model
python scripts/seed_baseline.py

# 5. Start backend
uvicorn backend.main:app --reload --port 8000

# 6. Start frontend (new terminal)
cd frontend && npm run dev    # → http://localhost:5173

# 7. Verify everything works
curl localhost:8000/health   # → {"status":"ok","redis":"connected"}
redis-cli ping               # → PONG
```